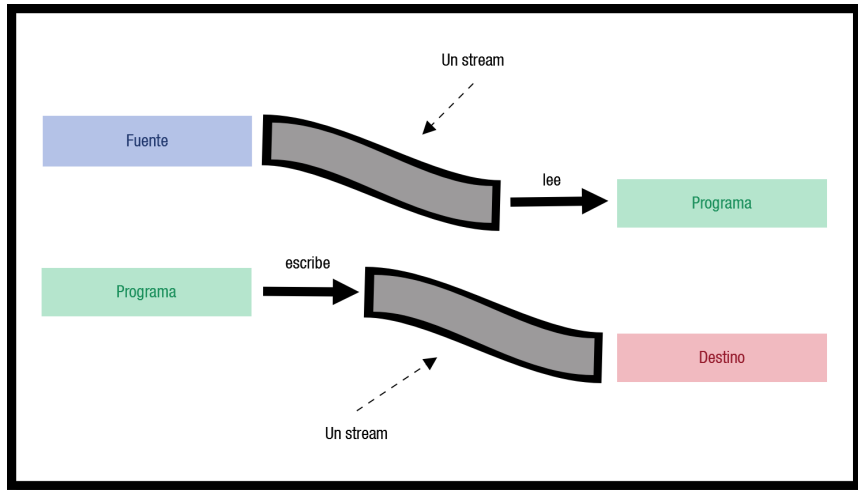


Flujo de datos

Un programa en Java, que necesita realizar una operación de entrada/salida (en adelante E/S), lo hace a través de un flujo o stream.



Las clases y métodos de E/S que necesitamos emplear son las mismas independientemente del dispositivo con el que estemos actuando, Java sabrá si tiene que tratar con el teclado, el monitor, un sistema de archivos o un socket de red; liberando al programador de tener que saber con quién está interactuando. Java define dos tipos de flujos en el paquete `java.io`:

1. **Streams binarios o de bytes (8 bits):** El tratamiento del flujo de bytes viene determinado por dos clases abstractas que son:
 - **InputStream:** Se utiliza para leer bytes desde una **fuentes** de datos. Clases:
 - **FileInputStream:** Lee bytes de un fichero.
 - **ObjectInputStream:** Convierte en objetos y variables los vectores de bytes leídos de un `InputStream`.
 - **OutputStream:** Se utiliza para escribir bytes a un **destino** de datos. Clases:
 - **FileOutputStream:** Escribe bytes en un fichero.
 - **ObjectOutputStream:** Convierte objetos y variables en vectores de bytes que pueden ser escritos en un `OutputStream`.
 - **DataOutputStream:** Da formato a los tipos primitivos y objetos `String`, convirtiéndolos en un flujo de forma que cualquier **DataInputStream**, de cualquier máquina, los pueda leer.
2. **Streams de caracteres o de texto (16 bits):** El tratamiento del flujo de texto viene determinado por dos clases abstractas que son:
 - **Reader:** Se utiliza para leer datos en forma de caracteres ó texto en lugar de bytes. Destacamos la subclase:
 - **FileReader:** lee caracteres de un fichero.
 - **Writer:** Se utiliza para escribir datos en forma de caracteres ó texto en lugar de bytes. Destacamos la subclase:
 - **FileWriter:** escribe caracteres en un fichero.

NOTA: La idea es que los accesos a disco sean los menos posibles, de aquí el uso de **Buffers**.

Si se usan sólo **FileInputStream**, **FileOutputStream**, **FileReader** o **FileWriter**, cada vez que se efectúa una lectura o escritura, se hace físicamente en el disco duro. Si se leen o escriben pocos caracteres cada vez, el proceso se hace costoso y lento por los muchos accesos a disco duro.

Los **BufferedReader**, **BufferedInputStream**, **BufferedWriter** y **BufferedOutputStream** añaden un buffer intermedio. Cuando se lee o escribe, esta clase controla los accesos a disco. Así, si vamos escribiendo, se guardarán los datos hasta que haya bastantes datos como para hacer una escritura eficiente. Al leer, la clase leerá más datos de los que se hayan pedido. En las siguientes lecturas nos dará lo que tiene almacenado, hasta que necesite leer otra vez físicamente. Esta forma de trabajar hace los accesos a disco más eficientes y el programa se ejecuta más rápido.

Además **FileReader** no contiene métodos que nos permitan leer líneas completas, pero sí **BufferedReader**.

Ejemplo 1: lectura de un fichero de texto con buffer

En este ejemplo leemos una línea completa del flujo.

```
File archivo = new File ("C:\\archivo.txt");
FileReader fr = new FileReader (archivo);
BufferedReader br = new BufferedReader(fr);
...
String linea = br.readLine();
```

Ejemplo 2: lectura de un fichero de texto con buffer

En este ejemplo leemos caracteres hasta llegar al fin de archivo (EOF o End Of File) del flujo.

```
import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
public class LeerFichEOF {
    public static void main (String args[]) throws IOException {
        BufferedReader br = new BufferedReader(new FileReader("origen.txt")) ;
        int codigo = br.read();//lee el primer caracter
        char caracter;
        while (codigo != -1) { //mientras el código no sea -1 (EOF) continuo leyendo
            caracter = (char) codigo; //casting
            System.out.print(caracter);
            codigo = br.read(); //lee un caracter
        }
    }
}
```

Vídeo sobre el paquete java.io.

<https://www.youtube.com/watch?v=2gWTVxe31g8>

Formas de acceso a un fichero

En Java puedes utilizar dos tipos de ficheros (de texto o binarios) y dos tipos de acceso a los ficheros:

1. **Acceso secuencial:** En este caso los datos se leen de manera secuencial, desde el comienzo del archivo hasta el final (el cual muchas veces no se conoce a priori). Este es el caso de la lectura del teclado o la escritura en una consola de texto, no se sabe cuándo el operador terminará de escribir.
2. **Acceso aleatorio:** Los archivos de acceso aleatorio, al igual que lo que sucede usualmente con la memoria (RAM=Random Access Memory), permiten acceder a los datos en forma no secuencial, desordenada. Esto implica que el archivo debe estar disponible en su totalidad al momento de ser accedido, algo que no siempre es posible. Java proporciona una clase:
 - **RandomAccessFile:** Permite leer y escribir sobre el fichero, no es necesario dos clases diferentes. Necesita que le especifiquemos el modo de acceso al construir un objeto de esta clase: sólo lectura o bien lectura y escritura. Posee métodos específicos de desplazamiento como `seek(long posicion)` o `skipBytes(int desplazamiento)` para poder movernos de un registro a otro del fichero, o posicionarnos directamente en una posición concreta del fichero. Por esas características que presenta la clase, un archivo de acceso directo tiene sus registros de un tamaño fijo o predeterminado de antemano.

A continuación puedes ver una presentación en la que se muestra cómo abrir y escribir en un fichero de acceso aleatorio. También, en el segundo código descargable, se presenta el código correspondiente a la escritura y localización de registros en ficheros de acceso aleatorio.

Ejemplo 1:

Vamos a ver un pequeño ejemplo, Log.java, que añade una cadena a un fichero existente, lo crea en caso de que no exista.

```
import java.io.IOException;
import java.io.RandomAccessFile;

public class RandomEjemplo {

    public static void main( String args[] ) throws IOException {

        RandomAccessFile miRAFile;

        String s = "linea a añadir al final del fichero";

        // Abrimos el fichero de acceso aleatorio

        miRAFile = new RandomAccessFile( "java.log","rw" );

        // Nos vamos al final del fichero

        miRAFile.seek( miRAFile.length() );

        // Incorporamos la cadena al fichero

        miRAFile.writeBytes( s );

        // Cerramos el fichero
```

```
    miRAFile.close();  
  }  
}
```